



Knowledge representation and reasoning in the context of automated production planning

Harkiran Sahota^{1,2} · Adam Kłodowski²

Received: 14 March 2024 / Accepted: 1 April 2025
© The Author(s) 2025

Abstract

In the manufacturing industry the step of generating the assembly plans for products is crucial but very time and labour intensive as most of the work has to be done manually. Especially for products with high customisability this means, a new plan is required for every custom order. To reduce the complexity, a new hybrid approach for the generation of these assembly plans is introduced, based on knowledge representation and reasoning. The focus of the approach is to be deployable in real-world production instead of only focusing on research environments. Therefore, a knowledge base is generated, specifying the preconditions and details about different operations linked to geometrical features. To preserve these valuable information Answer Set Programming (ASP) is used to define rules. By this, the generic information of when to apply which operation can be separated from the product-specific information of how a product looks, and therefore, be stored centrally and reused for any product or even different applications without any changes. Performing a geometric analysis on the product model allows extracting important geometrical features, that can then be used to derive the necessary information from the corresponding rules. In all of this, the human intervention is crucial to cover even complex assemblies and to strengthen the acceptance of the method. To showcase the functionality of this approach, a simple reproducible example is given on the assembly of the IKEA Hyllis shelf as well as the PUBLIC Bikes Sprout bike.

Keywords Automated assembly planning · Rule based system · Knowledge representation and reasoning · Answer set programming

Introduction

Manufacturing companies worldwide strive to realize the vision of Industry 4.0 (I4.0). Because of this the integration of modern technologies into product life cycles has become a critical priority (Kurasov, 2021). A morphological analysis of recent literature highlights the growing role of automation, AI, and data-driven decision-making in optimizing manufacturing workflows (Prashar, 2023). Driven by the demand for customizable, high-quality products at competitive prices, businesses are compelled to optimize every aspect of their operations, including production planning (Chaudron

et al., 2005; Sari, 2022). Among these, assembly planning for new or modified products stands out as a particularly complex and resource-intensive task. The growing diversity of product ranges and customization options only amplifies this challenge, often leaving companies reliant on manual planning processes that demand deep expertise in company-specific procedures and shop-floor dynamics. Relying on human effort not only limits efficiency but also introduces risks of inconsistency and error—barriers that hinder the scalability and adaptability required in a rapidly evolving industrial landscape. Consequently, there is an urgent need to digitize and automate assembly planning to enhance efficiency and adaptability in modern production environments while maintaining precision and flexibility (Wagner et al., 2024).

Recent studies have explored fully automated approaches to generate operation sequences for assembly planning (Deshmukh et al., 1993; Geoffrey, 2005; Knepper et al., 2013; Nägele et al., 2020). Although these efforts demonstrate the-

✉ Harkiran Sahota
Harkiran.Sahota@daimlertruck.com

Adam Kłodowski
Adam.Klodowski@lut.fi

¹ Daimler Buses, Neu-Ulm, Germany

² LUT University, Lappeenranta, Finland

oretical promise, their practical implementation often falls short. Full automation can fail in complex scenarios where incomplete knowledge leads to costly mistakes; therefore industrial acceptance remains low due to the “black-box” nature of such systems. Many of these solutions are validated in controlled, research-oriented settings with simplified examples, leaving a gap between academic success and real-world applicability. This disconnect motivates the current research: to bridge the divide between theoretical innovation and practical implementation in industrial environments. The primary contribution of this paper lies in addressing the following challenges:

1. the introduction of a hybrid approach to assembly planning that balances automation with human oversight,
2. the implementation of a centralized knowledge storage system to ensure data consistency by separating process and product knowledge,
3. the application of an Answer Set Programming-based rule base to capture, preserve, and leverage knowledge effectively.

These innovations are designed to enhance industrial applicability by enabling seamless integration into existing production systems, which is a critical step toward meeting the real-world demands of I4.0. This paper provides an overview of recent studies on assembly planning and Knowledge Representation and Reasoning methods, setting the stage for the proposed solution. The remainder of the paper is structured as follows. First, the assembly planning problem is defined, followed by an explanation of the new hybrid methodology. Two case studies are presented for the sake of clarity, accompanied by an analysis of the computational complexity of the approaches. The discussion and conclusion summarize the innovations, address limitations, and offer a roadmap for future research.

State of the art

In the first part of this section, recent studies in the field of assembly planning are introduced, analyzing different approaches. In the second part declarative and procedural approaches of Knowledge Representation and Reasoning (KRR) are presented, covering advancements and critiques of KRR approaches.

Analysis of assembly planning methodologies

In the paper by Nägele et al. (2020) a modular, two-player planning approach is presented on the LEGO bricks domain; planning the positioning and plugging of the bricks by multiple simultaneously working robots. The approach works

in three phases. First, in the structural analysis, a uniform product description is generated describing the existing relations and the desired goal state. Next, in the process planning phase, the product description is utilised to identify a potential workflow, by constantly comparing the current state of the product to the goal state while invalid workflows are filtered out. Finally, in the production planning phase, the found workflow is used to generate an automated solution for the robots by analysing their capabilities. For the optimization, the robots are scheduled in parallel for a faster assembly.

In the paper by Da Xu et al. (2012) a digital assembly planning and simulation system has been presented that has been tested on complex assemblies as aircraft components. The key functionality is its ability to automatically generate assembly plans from CAD models with minimal manual intervention. The system uses methods such as relationship matrices, assembly sequence planning, and path visualization to support the entire process from modelling to simulation.

In the paper by Knepper et al. (2013) a fully autonomous multi-robot furniture assembly process is presented using a combination of heterogeneous robots that perform distinct roles, such as delivering parts and assembling them. It starts by analyzing a user-provided geometric specification (CAD data) and uses this information to deduce how parts fit together. It generates a symbolic assembly plan that robots execute without further human intervention. In addition, robots can coordinate the manipulation of large, heavy parts during assembly, demonstrating the complex interaction between multiple robots.

In the paper by Bryson et al. (2009), a framework is introduced to capture and reuse knowledge in product development. Specifically, it focuses on creating a structured knowledge base that can be reused across different product development processes, integrating knowledge management practices in a practical engineering context. The emphasis is on technology reuse and transfer in a broad product life cycle from design, manufacturing, to the service. The objective is to ensure that past experiences and insights are systematically integrated into new projects.

In the paper by Spoor and Weber (2023), an approach is introduced that leverages neural networks, specifically Hopfield networks, for process planning in manufacturing. The neural network is trained on historical data of correct and incorrect process combinations, allowing it to learn patterns and optimize new process plans. After trained, the system can automatically generate efficient workflows for new product designs evaluating possible process configurations. The key advantage of this approach is its ability to adapt to dynamic workflows and make real-time planning adjustments. Similarly, in the paper by Xiao et al. (2024), an assembly planning pipeline utilizes a knowledge graph to extract and structure insights from historical process documents. This method enhances decision-making by integrating domain knowledge

with AI-driven planning, ensuring optimized and logically consistent assembly sequences. Combining this with neural networks could improve adaptability while maintaining structured reasoning in process planning.

The described methods show different approaches that can be applied to the domain of assembly planning. However, the approaches have significant limitations. The method by Nägele et al. (2020) focuses on a simplified assembly process, where all parts are identical in nature (bricks) and only two operations are considered: placing and plugging. It assumes that the connections and operations are straightforward and does not provide evidence that it can handle more complex products. While the possibility of expanding to other products is mentioned in the paper, it does not offer a methodology for adapting the system to more complex, heterogeneous assemblies. It is not further discussed how complex operations and constraints are managed. While the approach introduced by Da Xu et al. (2012) is aiming to minimize manual interventions, this can be a limitation if the system does not account for certain real-world constraints such as surface treatments or specialized operations. The system employs Assembly Relationship Matrices (ARM) to handle connections and interactions between parts. ARM is effective for predefined relationships and structured assemblies, but may lack the flexibility to manage diverse, non-standard parts without additional configuration. The approach by Knepper et al. (2013) automatically generates assembly sequences for coordinating multiple robots. This system focuses on robotic coordination instead of human worker instructions. The tool is highly automated but is specifically tailored to furniture assembly and may not be as flexible when applied to more varied or complex products that require detailed surface preparation or multiple types of connections. The approach by Bryson et al. (2009) does not explicitly focus on generating assembly instructions but aims for a broader knowledge reuse across multiple stages of product development. However, it emphasises designers and engineers to refer to knowledge from previous assembly instructions and best practices from similar products. The application of neural networks as described by Spoor and Weber (2023) and Xiao et al. (2024) allow for high adaptability. However, an effective training of neural networks or the generation of a knowledge graph requires a substantial amount of historical data, which many companies may not have readily available, hindering the widespread adoption of this approach.

Knowledge representation and reasoning

The key of KRR is to set up a knowledge base with a set of domain-specific predicates and functions. Effective knowledge representation and reasoning are critical for automated decision-making in production planning. The specific way of representing the knowledge depends on the concrete KRR

methods being used, some of which are described below. These methods enable intelligent systems to reason over complex assembly tasks, improving flexibility and adaptability in automated assembly planning (Levesque, 1986; Prashar, 2023).

Rule based expert system The most basic form of AI is the rule-based expert system (Talukdar et al., 2023). Rule based expert system is a wide concept that relies on human expertise to solve real-world problems by imitating human decision-making (Abraham, 2005). Therefore, the knowledge is provided by a human domain expert in the form of SITUATION-ACTION rules or IF-THEN rules (Gevarter, 1982). The collected human know-how on relevant information, rules, and relationships is then stored in the knowledge base and can be automatically applied to data (Abraham, 2005; Talukdar et al., 2023).

The typical rule-based system consists of three main components; the knowledge base long-term memory, the short term working memory and the inference procedure (Gevarter, 1982). While the long-term memory stores the human expertise in the form of rules, the short-term memory stores facts about the particular situation (Talukdar et al., 2023). The inference engine is the program that then performs the problem-solving using the data from the knowledge base and the working memory. Due to this clear separation of program and knowledge, the same knowledge base can be used with different inference engines and vice versa. The most common approach for the inference engine for decision-making are forward and backward chaining—applied separately or combined (Mukundan et al., 2007). The difference to conventional programs is its declarative form, describing the domain by rules and facts. In contrast to conventional programs, there are no procedural invocations, but rather the content of the working memory is being modified by the evoked rules, which in turn cause other rules to be invoked (Mukundan et al., 2007).

The advantage of a rule-based expert system is that the rather simple and uniform syntax that is used to formulate the rules allows to easily understand their meanings (Mukundan et al., 2007). Thus, a rule-based expert system allows capturing and preserving valuable expertise that would otherwise not be recorded in writing (Abraham, 2005). The modularity of the knowledge base also enables an easy extension of the rules, without affecting existing rules in any way. In conventional programming, the extension of a program by new procedures would require some adjustments to the existing program (Mukundan et al., 2007).

Prolog Prolog is a general purpose logic programming language that was invented in 1972, in Marseille, France. The name is an abbreviation for ‘PROgrammation en LOGique’. It was created to facilitate man–machine communication, employing natural language to describe the relations between things in an easy to read and write way (Colmerauer & Rous-

sel, 1996; Yang et al., 2022). Prolog only works with two elements, namely facts and rules that are stored in the Prolog database (Bramer, 2013). Providing the necessary facts and rules, the internal deductive reasoning mechanism is able to automatically solve the problem specified by the user (Yang et al., 2022).

A simple Prolog program could look like following:

```
lamp(on).
switch(X) :- lamp(X).
```

The first line is a fact, describing that *lamp* is *on*. The second line is a rule stating that any state (*X*) the *switch* is in, the *bulb* has the same state. The user entering the query

```
? - switch(on).
```

will receive the answer **true**, by the conclusion that *lamp* is *on* and *switch* is in the same state, therefore, it is true that *switch* is *on*.

Answer set programming Answer Set Programming (ASP) is a declarative KRR approach that is often used to model and solve complex search and combinatorial problems. It uses rules to justify or derive new information from given information. An ASP program is a finite set of rules, formed using literals (atoms and their negations). The body of a rule (right of $\leftarrow$) describes the preconditions needed to justify the head (left of $\leftarrow$) (Brewka et al., 2011). ASP produces a complete and optimal solution and although it offers a Prolog-like syntax it is guaranteed to terminate, unlike Prolog (Gambin et al., 2022).

An example ASP rule would look like:

```
light_on <- switch_on, not broken.
```

consisting of the three atoms *light_on*, *switch_on* and the negated *broken* and concluded by a period. As the closed-world assumption is adopted in ASP, the proposition is considered false, unless otherwise is stated (Aravanis et al., 2018). Accordingly, *light_on* is considered false unless *switch_on* is true and the *broken* is false. Again, both atoms are considered wrong unless otherwise declared. Therefore, the true atom has to be provided as a single fact, explicitly, as follows:

```
switch_on.
```

Given this fact, *switch_on* is now considered true while *broken* is still considered wrong and thus, according to the rule, *light_on* can be considered true. In general, the processing of these rules works in two steps. The rules are solved using a process named grounding, where the rules are instantiated with concrete values, replacing the variables (Brewka et al., 2011).

Model based reasoning Model Based Reasoning (MBR) is a special form of an expert system, particularly popular for troubleshooting systems. Waiting for a human expert to build the required expertise on a new system is often not advisable, as such systems become obsolete very fast. The idea behind MBR is to obtain the necessary knowledge by simply understanding the structure and functionality of a device. This is achieved by modeling a machinery or circuit by the components, their interconnections and interactions, to be able to find the reason behind faulty behaviour. Therefore, domain experts deliver basic rules about which symptoms are related to what kind of faults. The power of MBRs, however, is to diagnose new problems that have never occurred before by simply using the basic knowledge provided. This concept works especially well for electric circuit fault analysis, as their functionality is well-known and easy to model. However, it is difficult to apply this method in complex systems, as the question of how to model complex systems in the first place is still a topic of research (Mukundan et al., 2007).

Graph based approach In the graph-based approach, knowledge is represented as a graph, allowing to structure a massive amount of data. For the construction of a knowledge graph, raw data is processed using different methods for knowledge extraction, fusion and processing (Tian et al., 2022). It includes the recognition of named entities, their relationships and other attributes from raw data (Chen et al., 2022). This knowledge extraction can either be based on expert rules or neural network as convolutional neural networks (CNN) or recurrent neural networks (RNN). The generated graph is then stored in the knowledge base for further reasoning. As a knowledge graph usually suffers from missing information, knowledge reasoning is applied to discover missing connections. The reasoning can be based on simple rules, machine learning approaches or graph structure based knowledge inference (Tian et al., 2022). An example of such a knowledge graph is depicted in Fig. 1. It shows the different entities that are classified as objects or persons. The solid arrows indicate the relations between the entities. The dotted arrows indicate missing relations that can be reasoned upon the given information. For instance, the fact that Bob lives in a house with a room that has a burning lamp indicates that Bob has turned on the lamp.

Constraint programming Constraint programming is commonly applied to real-world combinatorial problems (Ben & Mouhoub, 2022; Van Beek & Chen, 1999). Constraints are commonly used for tasks in planning, scheduling, and optimisation (Hofstedt, 2010). It is required that the problem can be described as a constraint satisfaction problem (CSP). Therefore, a problem is modelled as $M = \langle X, D, C \rangle$, where X is a set of n variables $\{x_1, \dots, x_n\}$ that have to be instantiated to form the problem solution. D is a set of domains $\{D_1, \dots, D_n\}$, each describing possible values for the respective variable with $x_i \in D_i$. Finally, C is a set of constraints

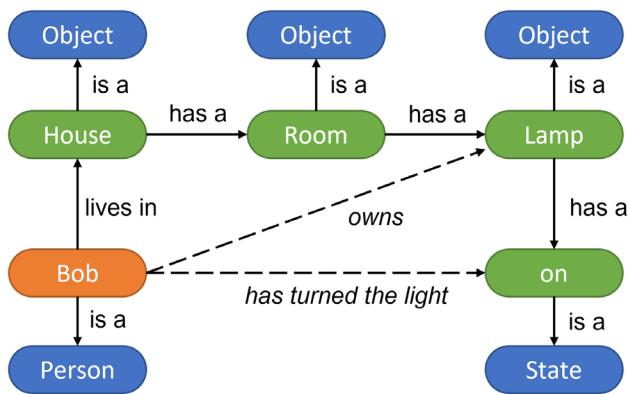


Fig. 1 Example of knowledge graph with entities and their relations. Dotted arrows indicate missing relations that can be derived by graph based reasoning

$\{C_1, \dots, C_m\}$ (Van Beek & Chen, 1999). A valid solution of M is an instantiation of all variables with values from their respective domains, whereby the instantiation does not violate any of the stated constraints (Ben & Mouhoub, 2022). To find such a solution, backtracking is commonly used in CSP. Thereby, in each step, a value from the respective domain is assigned to one variable. Afterwards, all constraints that include the newly assigned variable are checked for compliance and the domains of the remaining variables are adjusted to ensure the satisfiability. This way, the partial solution is extended in every step by the assignment of another variable (Van Beek & Chen, 1999).

As an example, consider $M = \langle X, D, C \rangle$, with

$$X = \{SWITCH, BULB, LIGHT, AMBIENT_LIGHT\}$$

and the variable domains

$$\begin{aligned} d_{SWITCH} &= \{ON, OFF\} \\ d_{BULB} &= \{NOT_BROKEN, BROKEN\} \\ d_{LIGHT} &= \{ON, OFF\} \\ d_{AMBIENT_LIGHT} &= \{NATURAL, ARTIFICIAL, NONE\} \end{aligned}$$

and given are the following constraints:

C1: $bulb = not_broken \text{ AND } switch = on \rightarrow light = on$

C2: $light = on \rightarrow ambient_light = artificial$

On the search for a valid solution, in the first step, the first variable $SWITCH$ will be assigned the value ON . To check the satisfiability of the constraints, $C1$ will be queued for review, as it contains the newly assigned variable $SWITCH$. As this constraint is still missing other variables, it can not be processed. Next, the $BULB$ variable will be assigned the

value NOT_BROKEN . Again, $C1$ would be queued for review as it contains the variable $BULB$. This time, processing the constraint will reduce the domain of $LIGHT$ to $\{ON\}$. Therefore, in the next step, $LIGHT$ will be assigned ON and the constraint $C2$ will be queued for review. Finally, the processing of this rule will lead to a reduction of the domain of $AMBIENT_LIGHT$ to $\{ARTIFICIAL\}$ which will then be assigned to the variable. So, the rules have led to a final assignment of the variables with

$$\begin{aligned} SWITCH &= ON \\ BULB &= NOT_BROKEN \\ LIGHT &= ON \\ AMBIENT_LIGHT &= ARTIFICIAL \end{aligned}$$

giving one possible solution for this CSP [cf. Van Beek and Chen (1999)].

Recent advancements in KRR

Hybrid approaches There is a recent trend in combining KRR methods with data-driven AI techniques like Reinforcement Learning (RL) and Machine Learning (ML). These approaches allow systems to adapt and learn from dynamic environments, improving their decision-making over time. A notable example is Yu et al. (2023) which discusses how these hybrid approaches improve reasoning tasks by incorporating learning from experience.

Scalability and efficiency A key focus in recent research such as by Gebser et al. (2013) are improving the scalability and efficiency of KRR methods, particularly for large-scale, complex domains such as industrial assembly. For example, improvements in ASP solvers, for example those developed in Clingo, offer efficient problem-solving for complex reasoning tasks, enabling their application in real-time industrial settings.

Explainability and interpretability As described by Balduccini and Gelfond (2003) KRR methods, especially rule-based systems such as ASP, offer transparent decision-making in which the reasoning process is comprehensible. This contrasts to black-box methods, such as AI methods, is making those methods particularly suited for domains where justifications are required.

Integration with robotics Researchers also focus on the integration of KRR methods into robotic systems for complex tasks, for example, assembly planning. Papers such as by Knepper et al. (2013) highlight the potential of using reasoning for task planning in dynamic, multi-robot environments. However, these systems often lack the flexibility needed for more complex industrial assemblies that are often performed manually.

Critiques in the field of KRR

Even though progress has been made, scalability remains a challenge. KRR systems can struggle to handle the increased complexity of large, multi-component assemblies in fields like automotive manufacturing. While ASP offers efficient reasoning, it can still face performance bottlenecks when applied to large datasets with numerous interdependent rules as stated by Gebser et al. (2013). A significant critique of KRR methods, especially rule-based systems, is the knowledge acquisition bottleneck as was already stated in the early paper by Felgenbaum (1977). Composing and maintaining rule sets can be time-consuming, requiring domain experts to capture the knowledge. This bottleneck can limit the practicality of deploying such systems in fast-evolving industrial environments. As noted by Erden and Gelfond (2016) KRR approaches as ASP, work well for predefined tasks but struggle with generalising to new, unseen tasks. This lack of flexibility means that significant reconfiguration is needed when applying these methods to different assembly scenarios, which limits their adaptability in dynamic production environments. KRR methods are often criticized for their limited capacity to handle uncertainty. While papers as by Baral et al. (2008) explore possibilities to extend ASP to incorporate probabilistic reasoning, traditional KRR systems still face challenges when dealing with incomplete or uncertain data.

Summary

In summary, recent advancements in KRR, as hybrid approaches and efficiency improvements, have made significant steps towards addressing scalability and adaptability. However, critiques remain regarding the knowledge acquisition bottleneck, generalisation issues, and limited handling of uncertainty.

Problem statement

A typical assembly consists of individual parts that are combined to one whole product, requiring fasteners: bolts, nuts and screws to fix the composition such as in Fig. 2. Typical features in such an assembly could be holes and the bolt that is inserted into it. Recognizing the screw or bolt connection and deriving the required instruction is straightforward for a human, thus a process plan can easily be formalized in natural language.

However, the manual process planning, as state of the art in many manufacturing companies today, requires experienced planners to identify the process steps in the 3D geometry of a product. The planners' knowledge, gained over years of working, is usually not documented sufficiently. Thus, many

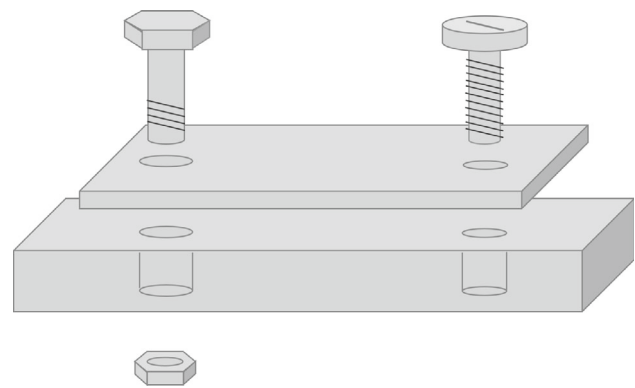


Fig. 2 Example of typical assembly, combining two parts by a bolt and nut (left) and a screw (right)

companies struggle from loss of knowledge when a planner leaves and face issues during the training of following planners. Furthermore, pure manual process planning is a time-consuming task, as a new process plan has to be generated for every new product. For high customizable products this means that every ordered product will require an own process plan according to the changes made by the customer. Another difficulty of the manual process planning is the natural language that is being used by the planners to describe the process. The natural language allows workers to easily understand the instructions, however, it is making efforts towards digitalization and automation more difficult. A company could process the assembly instructions to develop worker assistance systems, managing part availability at the work stations or even automate parts of the production. However, this requires a standardized language that is computer processable. A fully manual process planning complicates further processing, as different human planners use different ways of expressing the same instructions, each time. As many products are partially the same or at least contain similar connection types, planners help themselves by copying the instructions of similar existing products and making some modifications. This approach of manual effort reduction works because the required instructions are based on the parts or connection types of a product. Thus, similar products will have an instruction plan that will be similar at some points.

A more generic challenge is introduction of new approaches into existing production processes. Modifying established systems can be a costly and elaborate task, particularly when changes focus on a single aspect of production, such as assembly planning. The benefits of a new approach must clearly outweigh the significant costs involved in retraining personnel, adapting workflows, and updating equipment. If the new method only addresses planning without improving other key areas (e.g. execution, coordination, or quality control), the overall value may not justify the investment, leading

to reluctance from companies to adopt innovative technologies without a clear, comprehensive return on investment.

Methodology

To solve the issue of detailed instruction plan generation, a hybrid approach is developed to reuse knowledge in an automated way. This approach is based on the fact that specific assembly steps are directly connected to some specific features in the product and assembly, allowing to derive the correct set of operations by carefully analysing the product model. The method is based on the separation of the generic process knowledge and the specific product knowledge. Former contains rules about when a particular process has to be performed, while later contains the knowledge about how the product looks. The process knowledge is then used to derive the required instructions on the basis of the product knowledge, as depicted in Fig. 3. The possibility of making adjustments by the planner in an initial and final step are of great importance to cover complex scenarios and increase the acceptance in the production.

For this approach, first, the process knowledge has to be formalized. Therefore, a suitable knowledge base has to be created, and domain experts have to be consulted to generate the knowledge describing how or when an operation is performed. Here, KRR comes into play as it describes how to represent that knowledge and allow reasoning upon it. Despite the mentioned effort of generating and maintaining such a rule base this step is advantageous, as it facilitates the preservation of expert knowledge. Further, storing the expert knowledge can facilitate the usage of that information by other applications. The mentioned lack of flexibility requiring significant reconfiguration when applying KRR methods to different assembly scenarios is minor, as assembly processes in large manufacturing industries are not subject to frequent changes. The effort required for occasional adjustments is neglectable compared to the importance of knowledge preservation.

After studying possible methods it shows that ASP offers different advantages over the other covered KRR methods. ASP allows for a declarative way of describing what the problem is, making it easier to express and maintain complex relationships that can occur in assembly planning. In contrast, constraint programming focuses more on numeric constraints, while graph-based reasoning is designed for spatial and structural reasoning. Both are less effective for handling logical rules. Another benefit is the ASP solver which is highly optimised for solving complex combinatorial search problems. Both, model based reasoning and Prolog are not optimized for this purpose. Rule-based expert systems also lack efficiency when reasoning in large and complex domains. Thus, ASP seems to be the most promis-

ing KRR approach to write the rules. While Prolog or rule-based systems struggle with dynamic environments and changing knowledge or constraints, ASP is well-suited for non-monotonic reasoning. It is suited for situations where new information is introduced during the process as can happen in assembly planning. It is also important to mention that use of ASP also allows for the incorporation of probabilistic reasoning in future work, even if this is not yet relevant for the current approach. Due to these advantages ASP has been selected for representation and reasoning of the knowledge. Details about the terminology and computational complexities of the ASP solving can be found in the paper by Gebser et al. (2013).

To formalize the process knowledge in ASP, all assembly actions $A_1, \dots, A_x$ must be defined by the according constraints $C_1, C_2, \dots, C_n$ resulting in the logical condition:

$$A_y \rightarrow C_1 \wedge C_2 \wedge \dots \wedge C_n \quad (1)$$

The action A_y is triggered when all its constraints are satisfied. This relation can be described as a function:

$$f : C \rightarrow A \quad (2)$$

As an example, the rules for the screwing action, in their most basic form, could be described as:

```
screw_connection(Obj1, Obj2, S1) :-
    screw(S1, _, P1), hole(Obj1, P1), hole(Obj2, P1)
.

action("position", Obj1, Obj2) :-
    screw_connection(Obj1, Obj2, _) .

action("insert_screw", S1) :-
    screw_connection(Obj1, Obj2, S1),

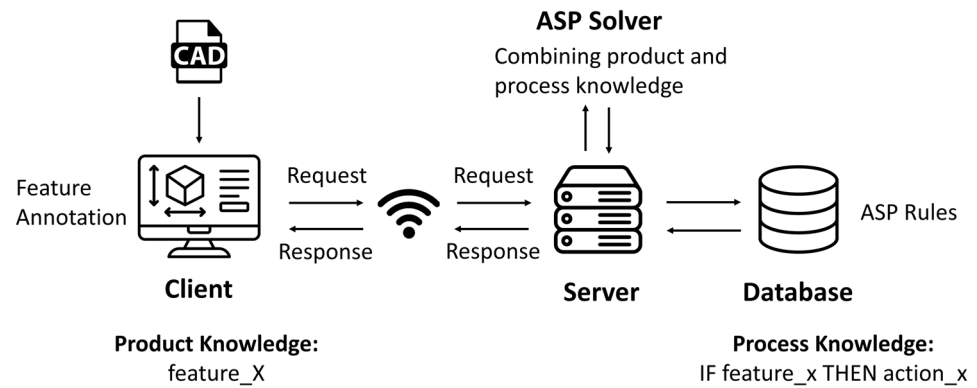
    action("position", Obj1, Obj2) .

action("tighten_loosely", S1, "hand") :-
    action(S1, "insert_screw"), .

action("tighten", S1, "Tool X") :-
    action(S1, _, "tighten_loosely"),
    screw(S1, Head_X, _) .
```

These rules describe a screw connection between *Obj1* and *Obj2* with screw *S1*. If both objects have a hole at the same position *P1* and a screw is at that position, a screw connection can be derived. Having a screw connection the action A_1 *position* can be derived. If there is a screw connection and the parts have been positioned, the action A_2 *insert screw* can be derived. If the screw *S1* has been inserted the action A_3 *tighten loosely* can be derived with the additional information to use the hands. If the screw *S1* has been tighten loosely and it has a specific screw head the action A_4 *tighten* can be derived

Fig. 3 Newly introduced hybrid approach. Separating the process knowledge and the product knowledge on server and client side. This should allow to reuse the general process knowledge even on other applications while tailoring the instructions according to the changing product knowledge and manual adjustments. Process and product knowledge are combined over the web service using the ASP solver



with the additional information to use the tool designated for that screw head.

These rules are very generic. Domain experts can specify more detailed rules by defining the required screw type, the torque, the right tool to use, how many workers the operation requires, and so on. To gather this information from the many experts in the different domains involved in the production of the specific product, a sequential approach is required. Successively extending the knowledge base bit by bit will require extensive planning and monitoring of the progress. As depicted in Fig. 3 the generated knowledge is stored centrally in a database accessed via a web interface to ensure that all users work with the most up-to-date information, even after modifications. This eliminates the risk of discrepancies in data across different local systems, as every planner or system accessing the knowledge base will retrieve the same rules and information. This centralization is crucial for maintaining consistency in assembly planning results across teams and locations. Additionally, hosting the knowledge in a central location enables it to be accessed by other applications, such as worker assistance systems, justifying the initial effort required to capture and maintain this knowledge. Centralizing the knowledge also enhances scalability, facilitating easier updates and integration as new tools or assembly processes are introduced.

For the actual planning, an application is to be made available to the planners as shown on the left side of the Fig. 3. After loading the CAD model using the standardised glTF format (The Khronos®3D Formats Working Group, 2021) the specific product knowledge is derived by extracting significant characteristics and features to reason upon them. Those product features will then serve to satisfy the *if* condition of the rules to derive further facts from the head giving the assembly instructions. In Fig. 4, a screw connection is depicted in a 3D geometry with all relevant features highlighted to serve the previously defined ASP rules. This includes the identification of the screw with its head type *HeadX* and the corresponding holes in the two objects, all at the same position *Position1*. Geometric feature annotation is a complex task, that is handled by the application using existing methods

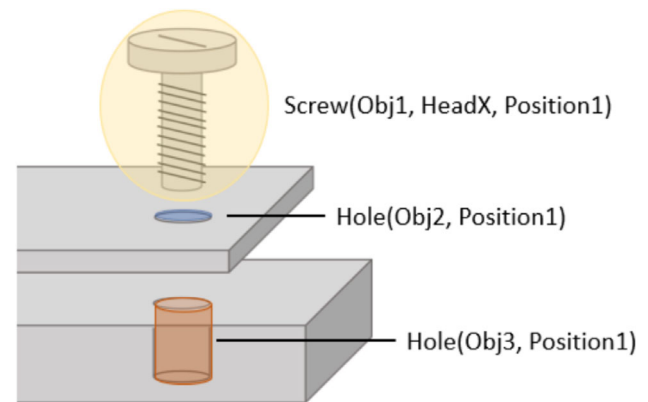


Fig. 4 Required annotations for a screw connection. Annotating the screw, and the corresponding screw holes with additional information as position or type of screw head

for automated geometry analysis. Nevertheless, only relying on the automated annotation of the models limits the applicability of the approach. Not all assembly instructions can be derived from the CAD models - glue points for example will never be included in the CAD file and will therefore not be detectable by geometry analysis methods. In these situations, the planner gets the possibility to manually add annotations through the application. Further, a revision of the automatically generated annotations can increase the acceptance of the approach in the company.

Which exact features have to be annotated, and at which level of detail, depends on the way the rules in the process knowledge have been defined. Again, the knowledge of the domain expert is the key in identifying which product features are related to which operations. In the example of the screw connection, it is required to annotate the screw with its position and head type and the holes in the objects. To utilize these annotations, the features and details have to be defined in a standardized way and expressed as ASP facts. For the example above the annotations in ASP look as following:

screw(s1, pos1, "headX").

hole(obj1, pos1).

hole(obj2, pos1).

For the derivation of the facts using these annotations and the rules from the rule base, an ASP solver is required. For this purpose the clingo by Potassco, the Potsdam Answer Set Solving Collection (Collection, Potassco - The Potsdam Answer Set Solving, 2025) is integrated into the web front (see Fig. 3). The application then transmits the generated facts about the model features to the web service which has access to the database with the previously defined ASP rules about the operations. The web services combines the product and process knowledge using the clingo solver to reason upon that information. The resulting instructions are then returned to the application. This way, the instructions can be generated in an automated way, while the knowledge is being preserved in rules. Also, applying the same rules will always lead to the same instruction, formalized in the same way, generating standardized plans.

Finally, the derived assembly instructions are visualized in the application for the planner to make final revisions and adjustments. Again, this is an important step to cover complex scenarios and strengthen the acceptance of the approach in the company.

Examples

To showcase the applicability of the previously described approach, a simple product with few assembly steps is selected. Therefore, the IKEA Hyllis shelf (Inter IKEA Systems B.V., 2013) was chosen. Further, to show the extendability of the presented approach, a second example is presented. The PUBLIC Sprout bike (PUBLIC Bikes, 2025b) is a simple assembly with few parts but allows to showcase how to reuse existing rules and apply the approach to new assemblies by simply extending the rule base.

Hyllis

The IKEA Hyllis shelf consists of a few distinct components and requires a simple assembly. Thus, this example allows to showcase the presented approach.

Problem description

The IKEA Hyllis shelf depicted in Fig. 5 consists of four legs, four shelf tops, 32 screws of the same type and 4 plastic stands—one for each leg. To assemble this shelf, the four legs have to be screwed to each shelf top using two screws (2 screws $\times$ 4 legs $\times$ 4 shelf tops = 32 screwing operations). However, before the individual screw can be screwed in, the parts have to be placed correctly—aligning the screw holes.

Fig. 5 IKEA Hyllis shelf (Inter IKEA Systems B.V., 2013) selected as a simple use case to highlight the applicability of the newly introduced approach on a simple assembly with few assembly steps



Finally, the 4 plastic stands have to be plugged into the bottom side of each leg, to prevent scratches on the floor.

Due to the simplicity of the chosen example, the assembly of the shelf consists of only two distinct actions - screwing and plugging in. Additionally, as the screw holes have to be correctly aligned, another action of positioning the parts is required. Following the pictorial assembly instructions given by IKEA, one possible assembly plan that could be derived is shown in Table 1.

Problem solving

Applying the proposed method of automated derivation of actions on the IKEA Hyllis example, first requires a database of according ASP rules. Investigating the given assembly plan in Table 1 rules have to be defined for the actions:

plug in, position, screw, flip

The *flip* operation is derived by the fact of accessibility of the screw holes on the ground - this requires the involvement of the environment and methods of collision detection. To minimize the complexity of the example, the environment and accessibility issues will be neglected for the time being. Therefore, a simplified assembly plan would look like stated in Table 2. To derive such an assembly plan in an automated way, the definition of rules for *plug in*, *position* and *screw* is required.

Position rule definition Positioning is required whenever a component is brought into its assembly position, prior to an assembly operation. Regardless of how the component will be attached to the assembly, it has to be brought into the right position, before it can be mounted with a subsequent operation, as screwing or nailing. If any of these joining operations are to be performed on the component, it must be preceded by a positioning operation. Thus, the following exemplary rules can be defined:

Table 1 One possible description of the assembly of the IKEA Hyllis, following the pictorial assembly instructions given by IKEA [cf. Inter IKEA Systems B.V. (2013)]

Step	Action	Component		Remark
1	Plug in stand on leg	Plastic leg	4x	Identify the bottom side of the leg
2	Position shelf 1 on leg 1			
3	Screw shelf 1 in leg 1	Screw	1x	Screw from side
4	Position shelf 2 on leg 1			
5	Screw shelf 2 in leg 1	Screw	1x	Screw from side
6	Position shelf 3 on leg 1			
7	Screw shelf 3 in leg 1	Screw	1x	Screw from side
8	Position shelf 4 on leg 1			
9	Screw shelf 4 in leg 1	Screw	1x	Screw from side
10	Position shelf 1 on leg 2			
11	Screw shelf 1 in leg 2	Screw	1x	Screw from side
12	Position shelf 2 on leg 2			
13	Screw shelf 2 in leg 2	Screw	1x	Screw from side
14	Position shelf 3 on leg 2			
15	Screw shelf 3 in leg 2	Screw	1x	Screw from side
16	Position shelf 4 on leg 2			
17	Screw shelf 4 in leg 2	Screw	1x	Screw from side
18	Position shelf 1 on leg 3			
19	Screw shelf 1 in leg 3	Screw	2x	Screw from side and top
20	Position shelf 2 on leg 3			
21	Screw shelf 2 in leg 3	Screw	2x	Screw from side and top
22	Position shelf 3 on leg 3			
23	Screw shelf 3 in leg 3	Screw	2x	Screw from side and top
24	Position shelf 4 on leg 3			
25	Screw shelf 4 in leg 3	Screw	2x	Screw from side and top
26	Position shelf 1 on leg 4			
27	Screw shelf 1 in leg 4	Screw	2x	Screw from side and top
28	Position shelf 2 on leg 4			
29	Screw shelf 2 in leg 4	Screw	2x	Screw from side and top
30	Position shelf 3 on leg 4			
31	Screw shelf 3 in leg 4	screw	2x	Screw from side and top
32	Position shelf 4 on leg 4			
33	Screw shelf 4 in leg 4	Screw	2x	Screw from side and top
34	Flip shelf			
35	Screw shelf 1 in leg 1	Screw	1x	Screw from top
36	Screw shelf 2 in leg 1	Screw	1x	Screw from top
37	Screw shelf 3 in leg 1	Screw	1x	Screw from top
38	Screw shelf 4 in leg 1	Screw	1x	Screw from top
39	Screw shelf 1 in leg 2	Screw	1x	Screw from top
40	Screw shelf 2 in leg 2	Screw	1x	Screw from top
41	Screw shelf 3 in leg 2	Screw	1x	Screw from top
42	Screw shelf 4 in leg 2	Screw	1x	Screw from top

```
position(Component_a) :-
    screw(Component_a,Position_x,Screw_x) ,
    screw(Component_b,Position_x,Screw_x) .
```

To further specify the given instructions, additional rules can be defined. In the case of the positioning process, this could be an instruction to describe the required tool. Depending on the size of the component it could be placed by hand, require a second worker or might be moved using a specific carrier. Therefore, the size of the components has to be given in a fact and compared to reference values (e.g. size equal or bigger than 100 requires a carrier, size between 10 and 100 requires two workers, size smaller than 10 is positioned by hand). Thus, the rules would be:

```
position_with(Component_a, "carrier") :-
    position(Component_a,Position_x),
    component(Component_a,_,Size_a),
    100<=Size_a .

position_with(Component_a, "two workers") :-
    position(Component_a,Position_x),
    component(Component_a,_,Size_a),
    10<=Size_a, Size_a<100 .

position_with(Component_a, "hand") :-
    position(Component_a,Position_x),
    component(Component_a,_,Size_a),
    10>Size_a.
```

Screw in rule definition A screw operation is performed whenever a component is assembled using screws. It should be noted, that there might be multiple numbers of screws required to mount one component to an assembly, requiring multiple screw operations on the same component, but in different locations. This will result in the rule:

```
screw(Component_a,Position_x,Screw_x) :-
    has_screw_hole(Component_a,Position_x),
    has_screw(Position_x, Screw_x) .
```

Again, the screwing operation can be further detailed by rules, for example specifying the required tool depending on the screw type as:

```
tighten_with(Component_a,Tool_x) :-
    screw(Component_a,_,Screw_1),
    screw_type(Screw_1,Type_x),
    tool(Tool_x, Type_x) .
```

Plug in rule definition Plug in is a special joint operation where the component is placed and mounted in one single operation, due to the special characteristics of the component. Performing such special joint operations, as plug in or gluing, include the operation of positioning. Thus, no separate position operation is required, though a separate, preceding

positioning operation would not distort the result and the shown representation is personal preference. Special characteristics of the component, such as a sticking surface or hooks to plug it in, indicate that this component is placed and mounted in one operation, such as:

```
plug_in(Component_a,Position_a) :-
    clip_component(Component_a),
    component(Component_a,Position_a,Size_a)
```

Once the required ASP rules are defined and stored in a database, these rules can be used to derive the instructions. To do so, facts about the assembly must be provided. This can be done by a geometry analysis to derive the facts about present screw holes, screw types, positions, etc. For the given example the provided facts have to look like following:

```
component(shelf1,pos_s1,10) .
component(leg1,pos_l1,15) .
component(stand1,pos_st1,3) .
screw_type(screw_1, type_a) .
tool("Phillips Screw Driver",type_a) .
clip_component(stand1).
screw(shelf1,pos_screw1,screw_1).
screw(leg1,pos_screw1,screw_1).
```

To generate the resulting instructions the open source answer set solver clingo (Collection, Potassco - The Potsdam Answer Set Solving, 2025) is used. Providing the defined rules and facts the following result is generated:

```
position(leg1) .
position(shelf1) .
position_with(shelf1,"two_workers") .
position_with(leg1,"two_workers") .
screw(leg1,shelf1,pos_screw1,screw_1) .
screw(shelf1,leg1,pos_screw1,screw_1) .
tool("Phillips Screw Driver",type_a) .
plug_in(stand1,pos_st1) .
position_with(stand1,"hand") .
```

These simple rules are already enough to deliver good results. However, further refinement of these rules can ensure a reasonable sequence of the instructions and prevent repetition of same operations (e.g. two screwing operations with switched parameters). The main goal of formulating these rules is to keep them as generic as possible, so they can easily be reused on different examples. If the shelf design changes, e.g. having less shelf tops, then the approach will still work as all necessary rules are available. Also, the rules are not tied to the specific shelf top and leg geometries. Thus, even the shelf depicted in Fig. 6 with less shelf tops and one different looking shelf top can be assembled using the defined rules, as long as the screw connections are present. In this case, no

Fig. 6 Altered version of the Hyllis shelf, with less shelf tops and a different looking shelf top. However, this version will not require any further efforts as all the rules for the required operations have been defined once and can be reused for the given example



extra effort for generating the new process plan is required for handling multiple product variants.

Sprout

The PUBLIC Bikes Sprout children's bike arrives as a kit, ready for the buyer to assemble. As such, the company includes comprehensive written assembly instructions, completed with illustrations, which form the foundation for the current case study (PUBLIC Bikes, 2025a). This instance illustrates the potential expansion of the proposed method.

Problem description

The Sprout bike as depicted in Fig. 7. It consists of 9 individual parts: frame, front fender, front wheel, seat, handle, two pedals and two training wheels. The bike frame is the base part to which all individual parts are added gradually. Fasteners such as nuts and bolts are already screwed in and require to be loosened or unscrewed completely before placing or adjusting the parts. This adds the maintenance-like context to this example (parts removal before installation of the new parts). Following the provided by the manufacturer instructions, one possible assembly plan could look as stated in Table 3.

Problem solving

To generate the assembly plan for the Sprout bike, a set of rules is required covering at least the following actions:

position, screw, unscrew, loosen, screw in, insert, push, adjust

As the rules for *position* and *screw* were already defined in the previous IKEA Hyllis example (section “Hyllis”) those rules can be reused in this example. However, the *position* rule require a little extension, as this example has washers that have to be positioned when screwing the according screw as:



Fig. 7 Sprout kids bike by PUBLIC. Delivered in individual components requiring assembly by adults (PUBLIC Bikes, 2025b)

```
position(Bolt_x, "place washer", Quantity) :-
    screw(_, Bolt_x),
    has_washer(Bolt_x, Quantity).
```

The new assembly mainly requires the definition of new rules for the actions: *unscrew*, *loosen*, *screw in*, *insert*, *push*, and *adjust* to deliver first satisfying results. Any next product would share more and more with the previous, so the knowledge quickly accumulates over time to a level where no new rules are required.

Unscrew rule definition Some parts are delivered with the screws and bolts already screwed in. Assembling another part to it requires the screw or bolt to be unscrewed and removed before positioning the other part. Later, the removed screw or bolt can be screwed in again using the already defined *screw* rule. However, it is not apparent from the geometry alone if screws are pre-installed in one part and have to be removed for the assembly first. Thus, this kind of annotation has to be added manually to the geometry enabling the *unscrew* actions to be derived from the rules. Further an addition to the *unscrew* rule can be added in case of washers to point out that the washers have to kept until reinstalling the bolt:

```
unscrew(Bolt_x, Position_x) :-
    screw(Bolt_x, Position_x),
    pre_positioned(Component_a, Bolt_X,
        Position_x).
```

```
hint("hold washer", Quantity) :-
    unscrew(Bolt_x, _),
    has_washer(Bolt_x, Quantity).
```

Similarly to the screw rule, the unscrew rule can be further detailed by specifying the required tool:

Table 2 Simplified version of the IKEA Hyllis assembly plan, neglecting the environment and accessibility issues (Inter IKEA Systems B.V., 2013)

Step	Action	Component		Remark
1	Plug in stand on leg	Plastic leg	4x	Identify the bottom side of the leg
2	Position shelf 1 on leg 1			
3	Screw shelf 1 in leg 1	Screw	2x	Screw from side and top
4	Position shelf 2 on leg 1			
5	Screw shelf 2 in leg 1	Screw	2x	Screw from side and top
6	Position shelf 3 on leg 1			
7	Screw shelf 3 in leg 1	Screw	2x	Screw from side and top
8	Position shelf 4 on leg 1			
9	Screw shelf 4 in leg 1	Screw	2x	Screw from side and top
10	Position shelf 1 on leg 2			
11	Screw shelf 1 in leg 2	Screw	2x	Screw from side and top
12	Position shelf 2 on leg 2			
13	Screw shelf 2 in leg 2	Screw	2x	Screw from side and top
14	Position shelf 3 on leg 2			
15	Screw shelf 3 in leg 2	Screw	2x	Screw from side and top
16	Position shelf 4 on leg 2			
17	Screw shelf 4 in leg 2	Screw	2x	Screw from side and top
18	Position shelf 1 on leg 3			
19	Screw shelf 1 in leg 3	Screw	2x	Screw from side and top
20	Position shelf 2 on leg 3			
21	Screw shelf 2 in leg 3	Screw	2x	Screw from side and top
22	Position shelf 3 on leg 3			
23	Screw shelf 3 in leg 3	Screw	2x	Screw from side and top
24	Position shelf 4 on leg 3			
25	Screw shelf 4 in leg 3	Screw	2x	Screw from side and top
26	Position shelf 1 on leg 4			
27	Screw shelf 1 in leg 4	Screw	2x	Screw from side and top
28	Position shelf 2 on leg 4			
29	Screw shelf 2 in leg 4	Screw	2x	Screw from side and top
30	Position shelf 3 on leg 4			
31	Screw shelf 3 in leg 4	Screw	2x	Screw from side and top
32	Position shelf 4 on leg 4			
33	Screw shelf 4 in leg 4	Screw	2x	Screw from side and top

```
unscrew_with(Bolt_x, Position_x, "wrench", SIZE) :-
    unscrew(Bolt_x, Position_x) ,
    component(Bolt_x, SIZE) .
```

Adjust rule definition Some parts require to be adjusted and placed carefully. This might be required for a precise alignment of parts before fully tightening to ensure proper fixation or meeting safety standards. It is not easily possible to recognise this kind of requirement from the geometry. Thus, a manual annotation is needed stating that a specific part has to be properly adjusted:

```
adjust(Component_a) :-
    adjust_component(Component_a) ,
    position(Component_a, Position_x) ,
```

```
screw_loosely(Component_a, Position_x,
    Screw_x) .
```

Note that a part can only be moved for adjustment if it has been screwed loosely. Thus, the existing *screw* rule has to be extended for components that require adjustment. In that case, the part may only be screwed loosely first and tightened in a second step to allow adjustments in between.

Loosen/tighten rule definition Some parts such as clamps require to loosen a screw and tighten it again later on. Geometry analysis can be used to recognize a clamp connection indicating that the clamp screw has to be loosened before the other part can be placed. Finally, the clamp screw has to be tightened again:

Table 3 One possible description of the assembly of the Sprout bike, following the textual assembly instructions given by PUBLIC Bikes [cf. PUBLIC Bikes (2025a)]

Step	Action	Component		Remark
Front fender installation				
1	Unscrew bolt on fork	Bolt Nut, washer (2x)	1x	10 mm wrench, hold bolt and washers
2	Position fender			
3	Screw fender to fork	Bolt, nut, washer (2x)	1x	Tighten lightly
4	Push fender			
5	Tighten nut	Nut	1x	
6	Unscrew bolt on fork	Bolt	2x	Left side 4 mm hex wrench
7	Position fender brace			Left side
8	Screw bolt on fork	Bolt	2x	Left side 4 mm hex wrench
9	Unscrew bolt on fork	Bolt	2x	Right side 4 mm hex wrench
10	Position fender brace			Right side
11	Screw bolt on fork	Bolt	2x	Right side 4 mm hex wrench
Front wheel installation				
12	Loosen nut	Front wheel	2x	15 mm wrench
13	Position Wheel	Front wheel		Washer on the outside
14	Tighten nut	Fork	2x	15 mm wrench
Insert stem and handle				
15	Insert handle			
16	Adjust handle	Handle bar		
17	Tighten bolt	Fork	2x	6 mm wrench
Pedal installation				
18	Screw in pedal			Left side
19	Screw in pedal			Right side
Adjust seat height				
20	Loosen clamp	Clamp bolt		5 mm hex wrench
21	Adjust seat			
22	Tighten clamp	Clamp bolt		5 mm hex wrench
Training wheel installation				
23	Unscrew bolt	Bolt, washer		15 mm box wrench Left side
24	Position plate	Metal plate		Left side
25	Position training wheel			Left side
26	Screw bolt	Bolt, washer		15 mm box wrench Left side
27	Unscrew bolt	Bolt, washer		15 mm box wrench Right side
28	Position plate	Metal plate		Right side
29	Position training wheel			Right side
30	Screw bolt	Bolt, washer		15 mm box wrench Right side

```

loosen_screw(Screw_s,Position_x) :-
  clamp_connection(Component_a,
    Component_b,Clamp_c) ,
  clamp_component(Clamp_c,Screw_s,
    Position_x) .

```

If a screw clamp has been loosened, it also has to be tightened later. So, after the parts have been positioned the clamp screw is tightened fully:

```

tighten_fully(Screw_s,Position_x) :-
  clamp_connection(Component_a,
    Component_b,Clamp_c) ,
  loosen_screw(Screw_s,Position_x) ,
  position(Component_b) .

```

Note that the position rule has to be extend - in case of a clamp connection the component might only be positioned after the clamp screw has been loosened.

Screw in rule definition This action is required for special parts that have a thread and are directly screwed into another part without requiring a separate screw. This connection can be detected by the geometry analysis by recognizing the thread on the part:

```

screw_in(Component_a,Component_b,Position_x) :-
  has_thread(Component_a,Position_X) ,
  screw_connection(Component_a,
    Component_b) .

```

Insert rule definition Another special rule for when a specific part is positioned inside another part, such as pipes, that are connected by inserting one into the other. This kind of connection can be recognized by geometry analysis:

```

insert(Component_a,Component_b,Position_x) :-
  stick_connection(Component_a,
    Component_b,Position_x) .

```

Using these rules and the previously defined *screw* and *position* rules, the assembly plan for the Sprout bike can be derived by providing the required facts. For the first part, front fender installation, those facts could look like following:

```

component(fork,pos_a,size_a) .
component(fender,pos_a,size_a) .
component(brace_left,pos_a,size_a) .
component(brace_right,pos_a,size_a) .
component(bolt_x,"10mm") .
component(bolt_y_left,"4mm") .
component(bolt_y_right,"4mm") .
screw(fork,pos_a,bolt_x) .
screw(fender,pos_a,bolt_x) .
screw(fork,pos_b,bolt_y) .
screw(brace_left,pos_b,bolt_y) .
screw(fork,pos_c,bolt_y) .
screw(brace_right,pos_c,bolt_y) .

```

```

screw(bolt_x,pos_a) .
has_washer(bolt_x) .
screw(bolt_y_left,pos_b_left) .
screw(bolt_y_right,pos_b_right) .
pre_positioned(frame,bolt_x,pos_a) .
pre_positioned(frame,bolt_y_left,pos_b_left) .
pre_positioned(frame,bolt_y_right,pos_b_right) .

```

producing the following outcome:

```

unscrew(bolt_x,pos_a) .
unscrew(bolt_y_left,pos_b_left) .
unscrew(bolt_y_right,pos_b_right) .
unscrew_with(bolt_x,pos_a,"wrench","10mm") .
unscrew_with(bolt_y_left,pos_b_left,
  "wrench","4mm") .
unscrew_with(bolt_y_right,pos_b_right,
  "wrench","4mm") .
position(fork) .
position(brace_left) .
position(brace_right) .
position(bolt_x,"washer","2x") .
screw(fork,pos_a,bolt_x) .
screw(brace_left,pos_b,bolt_y) .
screw(brace_right,pos_c,bolt_y) .

```

This part of the Sprout bike assembly already shows how the effort for a new assembly is reduced because already defined rules can be reused. Accordingly, the positioning, screwing and tighten_with actions were derived without writing any new rule. Only new actions require new rules. Thus, after a while the accumulated collection of rules will converge to cover any typical assembly.

Computational complexity

The assembly example of an IKEA shelf and a Sprout bike represent simpler scenarios compared to industrial-scale assemblies, such as those in the automotive sector. However, the methodology presented has been applied to complex, real-world bus assemblies, though detailed disclosure of these examples is restricted due to confidentiality agreements. Despite this, the computational complexity of the approach can be demonstrated through presented cases. To evaluate this, computation times were recorded for various assembly parts, segmented into three phases: first, the time required for loading and performing the geometric analysis of the CAD model (Phase I); second, the time for converting the extracted information into ASP facts (Phase II); and third, the time spent transmitting and receiving data from the web service, including the reasoning process executed on the server (Phase III). Furthermore, the maximum RAM

Assembly	Parts	Screws	Aux Parts	Phase I (sec)	Phase II (sec)	Phase III (sec)	RAM (MB)
Hand rail	23	17	40	12,2	5,1	26,6	1194
Air conditioning cover	27	108	176	90,6	358,3	31,4	1471
Cooling system	38	43	34	36,2	64,5	28,7	1555
Support Structure	40	15	54,0	33,3	9,1	28,5	985
Storage box	45	50	53	94,0	0,7	26,7	1297
Side door	117	28	164	81,4	162,4	30,1	1353
Front Bumper	121	84	242	137,7	239,7	31,7	1624

Fig. 8 Computational complexity of bus assemblies with different number of parts, screws and other auxiliary parts. Computation times (in seconds) divided into three phases: (I) loading and geometry analysis,

(II) information translation to ASP, (III) sending, reasoning and receiving of result. Stating the maximum RAM usage during runtime (in MB)

usage during the run time is analysed. The results of these measurements are provided in the following table.

An analysis of the recorded computation times indicates that the duration of the first phase increases proportionally with the number of components, as a greater number of parts necessitates more extensive geometric analysis. In contrast, the third phase consistently requires approximately 30 s, irrespective of the assembly's complexity or the number of components, suggesting that the web service's reasoning process is not significantly affected by the assembly size. However, in the second phase, high fluctuations in its values can be observed. This fluctuation arises from the current lack of optimization in the translation process. Although serving its purpose of converting the information into ASP facts, this phase could be improved to enhance processing speed, if required. Nevertheless, the time saving is considerable, as planners working with these assemblies have stated that the planning of these assemblies can take up to a few days depending on the number of parts.

Discussion and conclusion

The paper presents a hybrid approach to assembly planning explicitly aiming at better industrial applicability. This section summarizes the innovations, implications, and limitations of the presented approach and provides a final conclusion.

Innovations

The primary contribution of the proposed approach lies in its separation of production process knowledge from product descriptions. By leveraging Answer Set Programming (ASP), the system is designed to accumulate knowledge over time and remains extensible, adapting to evolving needs. At its core, the method relies on structured knowledge organization and logical reasoning, meeting industrial demands for a deterministic and scalable production process planning tool. The implementation emphasizes scalability and distributed

computing, marking a shift from research-focused experimentation to a practical, industry-oriented solution. In its current state, the approach enables a guided system tailored to industrial requirements. As it gathers sufficient knowledge over time and integrates semantically annotated parts from the design process, it has the potential to evolve into a fully autonomous system.

The separation of the rule base and the application is well thought-out. Providing the rule base via a web service increases the productive applicability as the knowledge is made available centrally while only the tool itself has to be distributed to the users. For a company, this means that the confidential knowledge can be stored in a safe database and the access can be regulated. The centralised access to the data ensures that every user works on the same data basis, even in the case of adjustments on the rules. Further, the separation of the rule base allows for better integration of the data into existing data management tools. This allows reuse of the rules in other applications such as worker assistance systems by offering specific interfaces. For an efficient industrial deployment the generated data should be accessible and usable for several applications to justify the efforts.

Recent approaches such as the one by Da Xu et al. (2012) or by Knepper et al. (2013) aim to minimize manual interventions by automating the assembly sequence generation. However, this reduces the flexibility of the approach. It can be a limitation if the system does not account for certain real-world constraints such as surface treatments or specialized operations. The presented approach does not aim for a fully automated generation of assembly plans but deliberately holds on to a combination of automated and manual inputs, making it more adaptable for complex, real-world manufacturing processes. This hybrid approach can be a strength, as it allows for detailed customization where full automation might struggle with complex tasks. Furthermore, the human revision can strengthen the acceptance of the approach in present production. This can be an important step towards a fully automated system in the future, when revised and approved data is gathered for backing those approaches. Nevertheless, it is considered that some human intervention

will always be required to change the knowledge or add new knowledge used by fully automated approaches. As soon as new machines, new tools or new working materials are introduced and significant changes in the assembly operations occur that cannot be derived from previous knowledge, humans will be required to provide the new information and safeguard its correctness.

Methods such as the Assembly Relationship Matrices (ARM) used by Da Xu et al. (2012) may be effective for predefined relationships and structured assemblies but may lack the flexibility to manage diverse, non-standard parts without additional configuration. Thus, it may face challenges in expanding beyond predefined assembly relationships. In contrast, ASP allows for more dynamic reasoning and adjustment of operations based on different products and their specific requirements. It provides a logical and flexible way to manage assembly operations based on features extracted from CAD models. Due to this, the new approach is more capable of handling real-world manufacturing processes where constraints such as order of operations, dependencies between parts, and resource allocation are critical.

While papers as by Nägele et al. (2020) introduce approaches that have been tested on simplified domains, such as the LEGO brick domain, the newly introduced approach is better suited to handle the diversity of real-world assembly scenarios, which require a more comprehensive understanding of the parts and operations involved. While the approach by Nägele et al. (2020) has only been tested on the LEGO bricks domain, the new approach has been validated with actual products, demonstrating its practicality and real-world applicability.

Implications

The application of the proposed approach for assisted assembly planning can have great impacts on manufacturing, especially when working with complex product. By partially automating steps of the assembly planning using geometry analysis and rule-based reasoning the time and effort required to generate an assembly plan can be significantly reduced as described in section “[Computational complexity](#)”. Especially for complex assemblies with many parts, the computation time of the proposed method hardly changes while the purely manual effort would notably increase. Further, the partial automation minimizes the potential for human error and enables a standardization in plan structure and wording across different products due to the application of predefined rules. Due to the modularity of the ASP rules, adaptations during an ever evolving manufacturing process are easy and companies can handle changes without significant reconfiguration. As ASP is designed to work efficiently with a complex rule and constraint base, it can handle sophisticated products well. This allows the method to be scalable and perform

well even with highly intricate assemblies. All these benefits, allowing for faster planning and less errors can ultimately lead to cost savings in the overall production process.

Implementation

The introduced approach is intended to be easily integratable into the existing production system. By utilising common data formats, such as the standardised geometry representation for the web: glTF format; the system can easily interface with the pre-existing design and manufacturing tool. Additionally, the rule-based system, implemented through Answer Set Programming (ASP), allows for straightforward adopting of existing production rules without significant changes to the workflow. The web-based service structure further enhances integration by enabling remote communication with the reasoning engine, ensuring that assembly planning can be performed without disrupting the existing production processes or requiring extensive system overhauls. This adaptability ensures smooth incorporation into the company’s production pipeline, requiring minimal modifications to current operations. However, the implementation of the proposed approach will cause a transition in the primary focus of the planners. The main focus will be on utilising the developed User Interface (UI) to review and refine the automatically generated annotations and the corresponding assembly instructions.

Limitations

Currently, the approach has been tested on a few but very common operations such as screwing or gluing. However, there is no limit to the number of integratable operations in applications. For a productive use it will require further work to establish a thorough database of ASP rules and extend the feature analysis function to cover all operations and improvement of the UI to offer all required functionalities allowing planners to effectively modify the generated plans.

As described in the section “[Methodology](#)” for some features the annotations must be added manually. This will initially increase the complexity and workload, however, the manual efforts will quickly decrease. As parts are often reused in new variations or products, the manual completion of the annotation of parts will be a one time effort. The reuse of the part within the product itself or in other products will also allow for the reuse of the annotations and thus the effort of manual annotations will decrease over time. Depending on the bus type one bus can already have 15–50% of repeated parts, which will decrease the manual annotation effort. Besides, not only many auxiliary parts as clips, rivets or fuses but also assembly parts in braking, engine or fuel systems are often reused as new variations or designs of the busses mostly change design parts or performance related parts while func-

tional parts remain the same. Same relations can be found in other industries as well.

In terms of testing the results, the current approach lacks consideration of the environment. A simulation of the generated assembly instructions would deliver better insights on the accessibility of parts. For example, it can be particularly important to observe the surroundings when assigning a tool to ensure that the space is sufficient to work with that tool. Otherwise, the space restrictions could require a special tool. Similarly, division into subassemblies has not been addressed in the current approach. It is however possible to achieve sub-assembly handling with additional rules for specific features or assembly scenarios.

As there may be periodic requirements to adjust the ASP rules, for instance, when new tools or operations are introduced, companies will need to address the long-term management of the ASP rule base. Even in the case of rare adaptations, there will be a need for proficiency in Answer Set Programming. Companies must determine whether it is more appropriate to provide training to some planners in ASP or to allocate IT specialists with ASP expertise to support the planners. Another possibility could be the consideration of developing another application to simplify the generation and adaption of ASP rules using a simple syntax or natural language. One hypothetical concept could be the use of Large Language Models (LLMs) to create ASP rules from natural language input. Using out of the box LLMs would reduce the cost for training and could deliver an easy way to work with ASP rules without any expertise on ASP. The capability of LLMs to generate ASP rules in the context of assisted assembly planning will be analysed in more detail in a future study.

Since some data, such as specific assembly instructions, might be confidential, ensuring secure handling of data in the ASP-based system becomes critical. Implementing security measures for sensitive data will be a key practical consideration for companies wanting to adopt this approach.

Conclusion

Nowadays, more and more modern technologies are being implemented into manufacturing processes to allow the production of high quality products at lower costs. One important field is assembly planning. It is required to formalize instructions for the workers for new or modified products. In many cases, this is still a very elaborate manual task. With the steadily increasing number of offered products and customizable variants, the digitalization and automation of this process should be considered. Therefore, a new hybrid approach based on knowledge representation and reasoning (KRR) is presented. The focus of this approach is to be easily integrat-

able into the existing production system instead of presenting an research environment-based approach only. This method relies on the division of generic knowledge about assembly operations and the specific information about each individual product. The generic knowledge is stored centrally to ensure the preservation of valuable information, a consistent knowledge base, and also to allow other applications to access the information. In a production environment this justifies the effort required to build the knowledge base. For this purpose the Answer Set Programming method is used to establish rules. Combining those rules with the product specific information about individual parts and connections allows to reason upon the given information and derive a tailored set of operations. As has been shown in the example, this allows to reuse the same set of rules and easily generate new instructions for a different product by simply adjusting the facts generated from the product features. In all this, human intervention in the form of revision and checking is facilitated using the provided application to cover even complex scenarios in production that cannot be derived from analysing the model only. Furthermore, there is a higher level of acceptance for hybrid approach with human revision than black-box, fully automated approaches, that are often criticized due to lack of accountability.

The implementation of the approach promises great benefits even if there are still some limitations. As an outlook, it is interesting to mention that the approach can further be expanded. Another benefit of ASP, that is yet to be incorporated, is the possibility to solve optimization problems. Instead of generating first possible solution, the rule base could be extended to optimize specific objectives in assembly planning, such as minimizing the time or the resource usage. Currently, no attention is paid to the physics throughout the assembly process. However, it is possible that the partially assembled product becomes unhandy for further processing or subsequent operations become infeasible because of accessibility reasons. These kinds of physics could be considered by further extending the rule base or by incorporating physics-based simulation in the results evaluation process. This would allow generating assembly sequence and adjusting the instructions if necessary.

Acknowledgements We would like to thank the CSC - IT Center for Science, Finland for the support in providing virtual server infrastructure in the project AIToC, that has been utilized in this project for data management system implementation.

Funding Open Access funding provided by LUT University (previously Lappeenranta University of Technology (LUT)).

Data availability Data supporting Tables 1 and 2 is openly available under Inter IKEA Systems B.V. (2013). Data supporting Table 3 is openly available under PUBLIC Bikes (2025a). The ASP datasets supporting the Use Cases in sections “Hyllis” and “Sprout” are openly available in the corresponding GitHub repository (Sahota & Klodowski, 2025) under the MIT License. The Hyllis 3D model files supporting this

study (GLB, GLTF, and JSON) are available from the authors upon reasonable request.

Declarations

Conflict of interest The authors have no conflict of interest to declare that are relevant to the content of this article.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Abraham, A. (2005). Rule-based expert systems. In *Handbook of measuring system design*. <https://doi.org/10.1002/0471497398.mm422>
- Aravanis, T., Demiris, K., & Peppas, P. (2018). Legal reasoning in answer set programming. In *IEEE 30th international conference on tools with artificial intelligence (ICTAI)* (pp. 302–306). <https://doi.org/10.1109/ICTAI.2018.00055>
- Balduccini, M., & Gelfond, M. (2003). Logic programs with consistency-restoring rules. In *International symposium on logical formalization of commonsense reasoning* (Vol. 102).
- Bär, T., & Bäuerle, D. (2022). Digital twin & digitalized bus manufactory. In *17. Fachkongress Digitale Fabrik, Nürnberg*.
- Baral, C., Gelfond, M., & Rushton, N. (2008). Probabilistic reasoning with answer sets. In *Theory and practice of logic programming* (Vol. 9). <https://doi.org/10.48550/arXiv.0812.0659>
- Ben Said, A., & Mouhoub, M. (2022). A constraint satisfaction problem (CSP) approach for the nurse scheduling problem. In *IEEE symposium series on computational intelligence (SSCI)* (pp. 790–795). <https://doi.org/10.1109/SSCI51031.2022.10022250>
- Bramer, M. (2013). *Logic programming with Prolog*. Springer. <https://doi.org/10.1007/978-1-4471-5487-7>
- Bratko, I. (2001). *Prolog programming for artificial intelligence (International computer science series)*. Addison Wesley. ISBN: 9780201403756.
- Brewka, G., Eiter, T., & Truszczyński, M. (2011). Answer set programming at a glance. *Communications of the ACM*, 54(12), 92–103. <https://doi.org/10.1145/2043174.2043195>
- Bründl, P., Scheffler, B., Stoidner, M., Nguyen, H., Baechler, A., Abrass, A., & Franke, J. (2023). Semantic part segmentation of spatial features via geometric deep learning for automated control cabinet assembly. *Journal of Intelligent Manufacturing*. <https://doi.org/10.1007/s10845-023-02267-1>
- Bryson, J., Cox, J. J., & Carson, J. T. (2009). A product development scenario for knowledge capture and reuse. *Computer-Aided Design and Applications*, 6(2), 207–218. <https://doi.org/10.3722/cadaps.2009.207-218>
- Chaudron, V., Martin, P., & Godot, X. (2005). Assembly sequences: Planning and simulating assembly operations. In *The 6th IEEE international symposium on assembly and task planning: From nano to macro assembly and manufacturing* (pp. 156–161). IEEE. ISBN: 0-7803-9080-6. <https://doi.org/10.1109/ISATP.2005.1511466>
- Chen, Y., Li, H., Li, H., Liu, W., Wu, Y., Huang, Q., & Wan, S. (2022). An overview of knowledge graph reasoning: Key technologies and applications. *Journal of Sensor and Actuator Networks*. <https://doi.org/10.3390/jsan11040078>
- Collection, Potassco - The Potsdam Answer Set Solving. (2025). Clingo and gringo. <https://potassco.org/clingo/> (visited on 03/24/2025).
- Colmerauer, A., & Roussel, P. (1996). The birth of Prolog. In *History of programming languages—II* (pp. 331–367). Association for Computing Machinery. ISBN: 0201895021. <https://doi.org/10.1145/234286.1057820>
- Da Xu, L., Wang, C., Bi, Z., & Yu, J. (2012). AutoAssem: An automated assembly planning system for complex products. *IEEE Transactions on Industrial Informatics*, 8(3), 669–678. <https://doi.org/10.1109/TII.2012.2188901>
- Da Xu, L., Wang, C., Bi, Z., & Yu, J. (2014). Object-oriented templates for automated assembly planning of complex products. *IEEE Transactions on Automation Science and Engineering*, 11(2), 492–503. <https://doi.org/10.1109/TASE.2012.2232652>
- Daimler Truck AG. (2023a). Daimler buses. <https://www.daimlertruck.com/unternehmen/daimler-buses> (visited on 03/24/2025).
- Daimler Truck AG. (2023b). Herzlich Willkommen bei Daimler Truck. <https://www.daimlertruck.com/unternehmen> (visited on 03/24/2025).
- Deshmukh, A., Paul Yung, J., & Wang, H.-P. (1993). Automated generation of assembly sequence based on geometric and functional reasoning. *Journal of Intelligent Manufacturing*, 4, 269–284. <https://doi.org/10.1007/BF00124140>
- Ding-yi, Z., Peng, W., Yan-li, Q., Lin-shen, F. (2019). Research on intelligent manufacturing system of sustainable development. In *2nd world conference on mechanical engineering and intelligent manufacturing (WCMEIM)* (pp. 657–660). <https://doi.org/10.1109/WCMEIM48965.2019.00139>
- Dong, T., Ruofeng, T., Ling, Z., & Jinxiang, D. (2007). A knowledge-based approach to assembly sequence planning. *The International Journal of Advanced Manufacturing Technology*, 32, 1232–1244. <https://doi.org/10.1007/s00170-006-0438-1>
- Erden, E., & Gelfond, N. L. (2016). Applications of ASP in robotics. In *AI magazine* (Vol. 37). <https://doi.org/10.1007/s13218-018-0544-x>.
- Fazio, T.L. de, Edsall, A. C., Gustavson, R. E., Hernandez, J. A., Hutchins, P. M., Leung, H.-W., Luby, S. C., Metzinger, R. W., Nevins, J. L., Tung, K. K., & Whitney, D. E. (1991). A prototype of feature-based design for assembly. In *Computer-aided cooperative product development. Lecture notes in computer science* (Vol. 492, pp. 369–392). Springer. ISBN: 978-3-540-54008-3. <https://doi.org/10.1007/BFb0014287>
- Felgenbaum, E. A. (1977). The art of artificial intelligence: Themes and case studies of knowledge engineering. In *Proceedings of the 5th international joint conference on artificial intelligence—Volume 2. IJCAI'77* (pp. 1014–1029). Morgan Kaufmann Publishers Inc.
- Gamblin, T., Massimiliano, C., Gregory, B., & Sergei, S. (2022). Using answer set programming for HPC dependency solving. In *SC22: International conference for high performance computing, networking, storage and analysis* (pp. 1–15). <https://doi.org/10.1109/SC41404.2022.00040>
- Gebser, M., Kaminski, R., Kaufmann, B., & Schaub, T. (2013). *Answer set solving in practice*. Springer. ISBN: 978-3-031-00433-9. <https://doi.org/10.1007/978-3-031-01561-8>
- Geoffrey, B. (2005). *Assembly automation and product design*. CRC Press. ISBN: 978-1-57444-643-2.
- Gevarter, W. B. (1982). An overview of expert systems. <https://ntrs.nasa.gov/citations/19820022023> (visited on 03/24/2025).

- Hofstedt, P. (2010). Constraint-based object-oriented programming. *IEEE Software*, 27(5), 53–56. <https://doi.org/10.1109/MS.2010.89>
- Hu, X., & Bo, G. (2018). Research on assembly sequence planning based on fluid flow mechanism. In *IEEE international conference* (pp. 46–50). <https://doi.org/10.1109/IRCE.2018.8492915>
- Inter IKEA Systems B.V. (2013). HYLLIS. https://www.ikea.com/de/de/assembly_instructions/hyllis-regal-drinnen-draussen_AA-1001159-9-2.pdf (visited on 03/24/2025).
- Knepper, R., Layton, A. T., Romanishin, J., & Rus, D. (2013). IkeaBot: An autonomous multi-robot coordinated furniture assembly system. In *IEEE international conference on robotics and automation* (pp. 885–862). <https://doi.org/10.1109/ICRA.2013.6630673>
- Kurasov, D. A. (2021). Computer-aided manufacturing: Industry 4.0. In *IOP conference series: Materials science and engineering* (Vol. 1047, No. 1, p. 012153). <https://doi.org/10.1088/1757-899X/1047/1/012153>
- Lanham, J., & Farid, D. (2001). The assembly state vector: A new approach to the generation of assembly sequences. In: *Proceedings of the 2001 IEEE international symposium on assembly and task planning (ISATP2001). Assembly and disassembly in the twenty-first century* (pp. 37–42). IEEE. ISBN: 0-7803-7004-X. <https://doi.org/10.1109/ISATP.2001.928963>
- Levesque, H. J. (1986). Knowledge representation and reasoning. *Annual Review of Computer Science*, 1(1), 255–287. <https://doi.org/10.1146/annurev.cs.01.060186.001351>
- Mukundan, S., Ramani, S., Raman, S., Anjaneyulu, K. S. R., & Chandrasekar, R. (2007). *A practical introduction to rule based expert systems*. Narosa Publishing House.
- Nägele, L., Schierl, A., Hoffmann, A., & Reif, W. (2020). LegoBot: Automated planning for coordinated multi-robot assembly of LEGO structures. In *IEEE/RSJ international conference on intelligent robots and systems (IROS)*. <https://doi.org/10.1109/IROS45743.2020.9341428>
- Prashar, A. (2023). Production planning and control in industry 4.0 environment: A morphological analysis of literature and research agenda. *Journal of Intelligent Manufacturing*, 34, 2513–2528. <https://doi.org/10.1007/s10845-022-01958-5>
- PUBLIC Bikes. (2025a). Sprout assembly instructions. https://cdn.shopify.com/s/files/1/0021/1399/4815/files/Sprout_assembly_instructions_updated_v7.pdf?v=1595434960 (visited on 03/24/2025).
- PUBLIC Bikes. (2025b). Sprout C 16. <https://publicbikes.com/products/sprout-c-16> (visited on 03/24/2025).
- Qian, J., Zhijing, Z., Chao, S., Hanqing, G., & Dongmei, L. (2021). Assembly sequence planning method based on knowledge and ontostep. *Procedia CIRP*, 97, 502–507. <https://doi.org/10.1016/j.procir.2020.05.266>
- Sahota, H., & Klodowski, A. (2025). Knowledge representation and reasoning in the context of automated production planning. <https://github.com/harkiransahota/Knowledge>
- Sari, T. (2022). The relationship between the implementation levels of industry 4.0 technologies and advanced manufacturing technologies. *Management and Production Engineering Review*, 13(3), 52–60. <https://doi.org/10.24425/mper.2022.142382>
- Smit, J., Kreutzer, S., Moeller, C., & Carlberg, M. (2016). *Policy Department A: Economic and scientific policy. Industry 4.0*. Online. Analytical Study. [https://www.europarl.europa.eu/RegData/etudes/STUD/2016/570007/IPOL_STU\(2016\)570007_EN.pdf](https://www.europarl.europa.eu/RegData/etudes/STUD/2016/570007/IPOL_STU(2016)570007_EN.pdf) (visited on 02/08/2023).
- Spoor, J. M., & Weber, J. (2023). Evaluation of process planning in manufacturing by a neural network based on an energy definition of hopfield nets. *Journal of Intelligent Manufacturing*, 35, 2625–2643. <https://doi.org/10.1007/s10845-023-02158-5>
- Talukdar, J., Singh, T. P., & Barman, B. (2023). Rule-based expert systems. In *Artificial intelligence in healthcare industry*. Springer Nature Singapore. https://doi.org/10.1007/978-981-99-3157-6_8
- The Khronos®3D Formats Working Group. (2021). <https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html> (visited on 09/27/2024).
- Tian, L., Zhou, X., Wu, Y.-P., Zhou, W.-T., Zhang, J.-H., & Zhang, T.-S. (2022). Knowledge graph and knowledge reasoning: A systematic review. *Journal of Electronic Science and Technology*, 20(2), 100159. <https://doi.org/10.1016/j.jnlest.2022.100159>
- Van Beek, P., & Chen, X. (1999). CPlan: A constraint programming approach to planning. In *Proceedings of the sixteenth national conference on artificial intelligence and the eleventh innovative applications of artificial intelligence conference innovative applications of artificial intelligence* (pp. 585–590).
- Wagner, S., Gonnermann, C., Wegmann, M., Listl, F., Reinhart, G., & Weyrich, M. (2024). From framework to industrial implementation: The digital twin in process planning. *Journal of Intelligent Manufacturing*. <https://doi.org/10.1007/s10845-023-02268-0>
- Xiao, Y., Zheng, S., Leng, J., Gao, R., Fu, Z., & Hong, J. (2024). An assembly process planning pipeline for industrial electronic equipment based on knowledge graph with bidirectional extracted knowledge from historical process documents. *Journal of Intelligent Manufacturing*. <https://doi.org/10.1007/s10845-024-02423-1>
- Yang, F., Xie, J., & Zhong, Z. B. (2022). Analysis on the application of intelligent robots based on Prolog language in primary and secondary education in the era of digital intelligence. In *3rd international conference on education, knowledge and information management (ICEKIM)* (pp. 305–309). <https://doi.org/10.1109/ICEKIM55072.2022.00075>
- Yu, C., Zheng, X., Zhuo, H. H., Wan, H., & Luo, W. (2023). Reinforcement learning with knowledge representation and reasoning: A brief survey. *arXiv Preprint*. <https://doi.org/10.48550/arXiv.2304.12090>
- Yuheng, W., Fodor, P., & Kifer, M. (2023). Knowledge authoring for rules and actions. *Theory and Practice of Logic Programming*, 23(4), 797–811. <https://doi.org/10.1017/S1471068423000169>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.